



UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE AND TECHNOLOGY
SCHOOL OF ICT
DEPARTMENT OF COMPUTER SCIENCE

Parallel and Distributed Computing

Distributed Systems Assignment Lab
Introduction to Socket Programming

Group Members Participation Table

Row No.	Registration No.	Names
1	222004611	Clement MUGISHA
2	222014353	Marie Mireille Irafasha
3	222005615	Joseph Bonheur Iradukunda
4	222002123	Rose Umutesi
5	222019837	Izere Bugingo Vainqueur Beryl
6	222002227	Elvis Mugisha
7	222005152	Uwineza Florance
8	222018513	Mugisha Edson
9	222004590	Sugira Herve
10	222008836	Marie Claire Nisingizwe
11	_____	_____

May 21, 2026

1. What is Socket Programming?

A **socket** is one endpoint of a two-way communication link between two programs running on a network. A socket is bound to a port number so that the transport layer can identify the application to which data is destined. Sockets are the fundamental building blocks of network communication — they allow processes on different machines (or the same machine) to exchange data as if they were writing to and reading from a file.

Socket programming is the practice of writing programs that use sockets to communicate over a network. Python exposes this through the built-in **socket** module, which provides a low-level interface directly to the operating system's networking stack.

1.1 TCP vs UDP

There are two dominant transport-layer protocols used with sockets:

TCP (Transmission Control Protocol) — connection-oriented, reliable, ordered, and error-checked. Before any data is exchanged, a connection is established through a three-way handshake (SYN → SYN-ACK → ACK). Every byte sent is guaranteed to arrive in the correct order. This makes TCP ideal for applications where data integrity matters, such as our directory lookup service.

UDP (User Datagram Protocol) — connectionless, unreliable, and unordered. Packets (datagrams) are fired without establishing a connection and may arrive out of order or not at all. UDP is preferred when speed matters more than reliability, such as video streaming or online games.

This lab uses **TCP sockets** exclusively, matching Python's `socket.SOCK_STREAM` type.

1.2 The Client-Server Model

Socket-based applications almost always follow the **client-server model**:

- The **server** is a long-running process that binds to a fixed address and port, listens for incoming connections, and serves requests.
- The **client** is a shorter-lived process that knows the server's address and port, initiates the connection, sends requests, and reads responses.

In our lab, `server/server.py` is the server and `client/client.py` is the client. The server holds a database of UR-CST students and staff. Clients connect and query that database over TCP.

2. Lab Overview — UR-CST Staff/Student Directory

The School of ICT at the University of Rwanda — College of Science and Technology (UR-CST) has four departments sharing a central server:

- Department of Computer Engineering
- Department of Computer Science

- Department of Information Systems
- Department of Information Technology

The central server stores records for every student and employee. Each record contains: department number, first name, last name, phone number, and email address. Any client (student or teacher) can connect to the server and make one of the following queries:

- **List all records** — retrieve the full directory.
- **Email by first name + last name** — look up a person's email directly.
- **Email by last name + department number** — useful when only partial information is known.
- **Phone by first name + last name** — retrieve a phone number.
- **List all members in a department** — retrieve everyone in a given department.

3. The Server — Binding, Listening, and Accepting Connections

The server side of a TCP socket application goes through four distinct steps before it can serve any client: **create**, **bind**, **listen**, and **accept**.

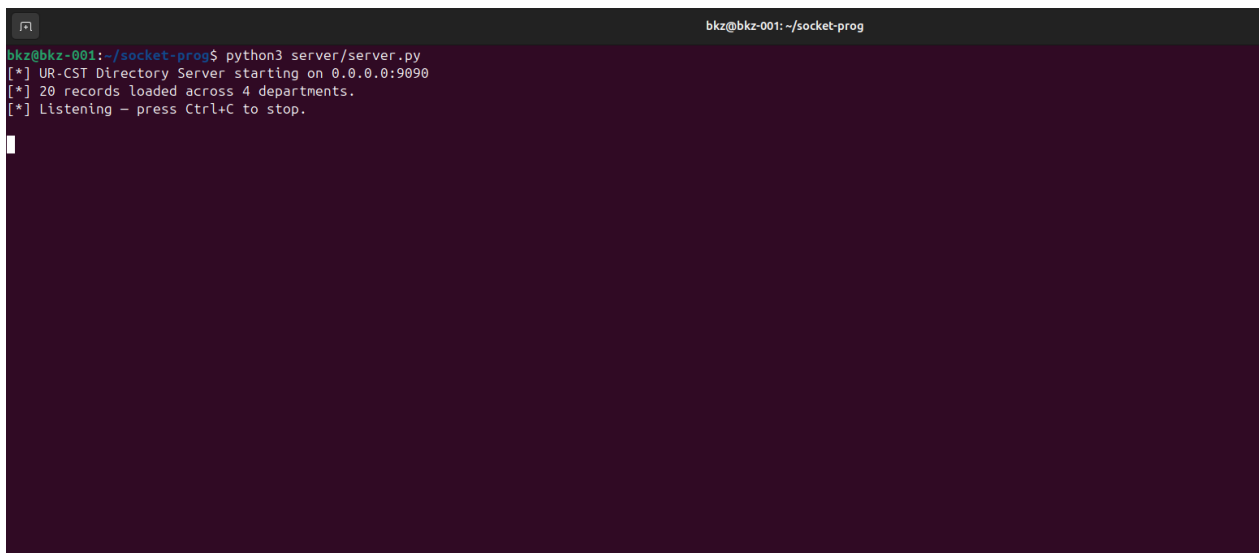
3.1 Creating and Binding the Socket

The server creates a TCP socket, sets the `SO_REUSEADDR` option so the port can be reused immediately after a restart, then binds to `0.0.0.0:9090`. Binding to `0.0.0.0` means the server listens on all available network interfaces.

```
srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("0.0.0.0", 9090))
srv.listen(10)
```

3.2 Server Startup — Screenshot 1

Running `python server/server.py` produces the following startup output, confirming the socket is bound and listening:

A terminal window with a dark background and light green text. The prompt is 'bkz@bkz-001: ~/socket-prog'. The user has run 'python3 server/server.py'. The output shows: '[*] UR-CST Directory Server starting on 0.0.0.0:9090', '[*] 20 records loaded across 4 departments.', and '[*] Listening - press Ctrl+C to stop.'

```
bkz@bkz-001:~/socket-prog$ python3 server/server.py
[*] UR-CST Directory Server starting on 0.0.0.0:9090
[*] 20 records loaded across 4 departments.
[*] Listening - press Ctrl+C to stop.
```

SS1 — Server starts, binds to 0.0.0.0:9090, and enters the listening state.

3.3 Accepting Connections

`srv.listen(10)` sets the backlog — the maximum number of queued connections waiting to be accepted. The server then loops forever, calling `srv.accept()`, which blocks until a client connects and returns a new socket object dedicated to that client along with the client's address.

```
while True: conn, addr = srv.accept() t = threading.Thread(target=client_handler,
args=(conn, addr), daemon=True) t.start()
```

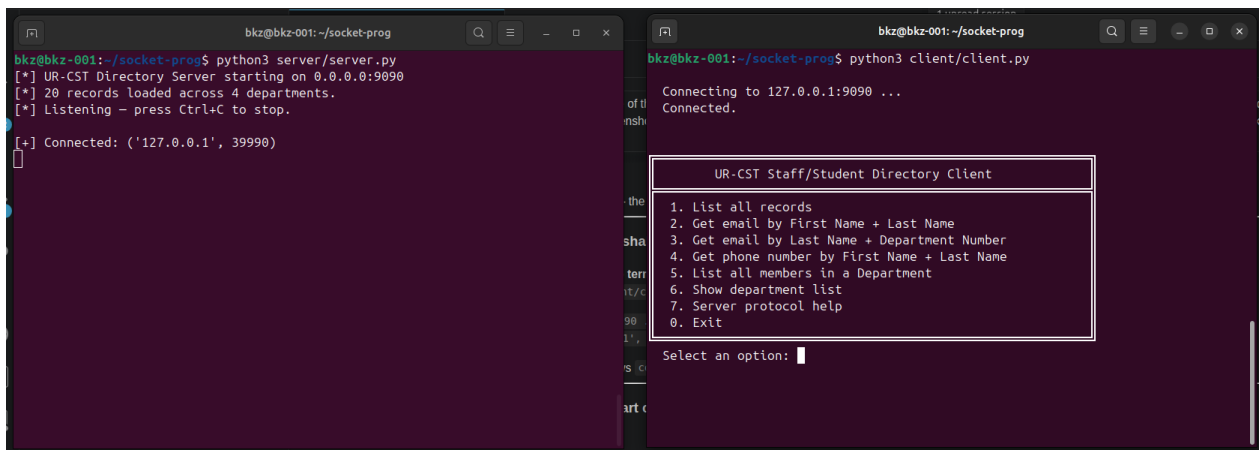
4. The Client — Connecting to the Server

The client creates the same type of TCP socket and calls `connect()` with the server's IP address and port. This triggers the TCP three-way handshake. A 5-second timeout is set before connecting so the client fails fast if the server is unreachable.

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) sock.settimeout(5)
sock.connect(("127.0.0.1", 9090)) sock.settimeout(None) # restore blocking mode
after connect
```

4.1 The Connection Moment — Screenshot 2

The screenshot below shows both terminals side by side at the instant a client connects. The server terminal (left) logs the client's address. The client terminal (right) confirms the connection and displays the menu.



```
bkz@bkz-001:~/socket-prog$ python3 server/server.py
[*] UR-CST Directory Server starting on 0.0.0.0:9090
[*] 20 records loaded across 4 departments.
[*] Listening - press Ctrl+C to stop.

[+] Connected: ('127.0.0.1', 39990)

bkz@bkz-001:~/socket-prog$ python3 client/client.py
Connecting to 127.0.0.1:9090 ...
Connected.

UR-CST Staff/Student Directory Client

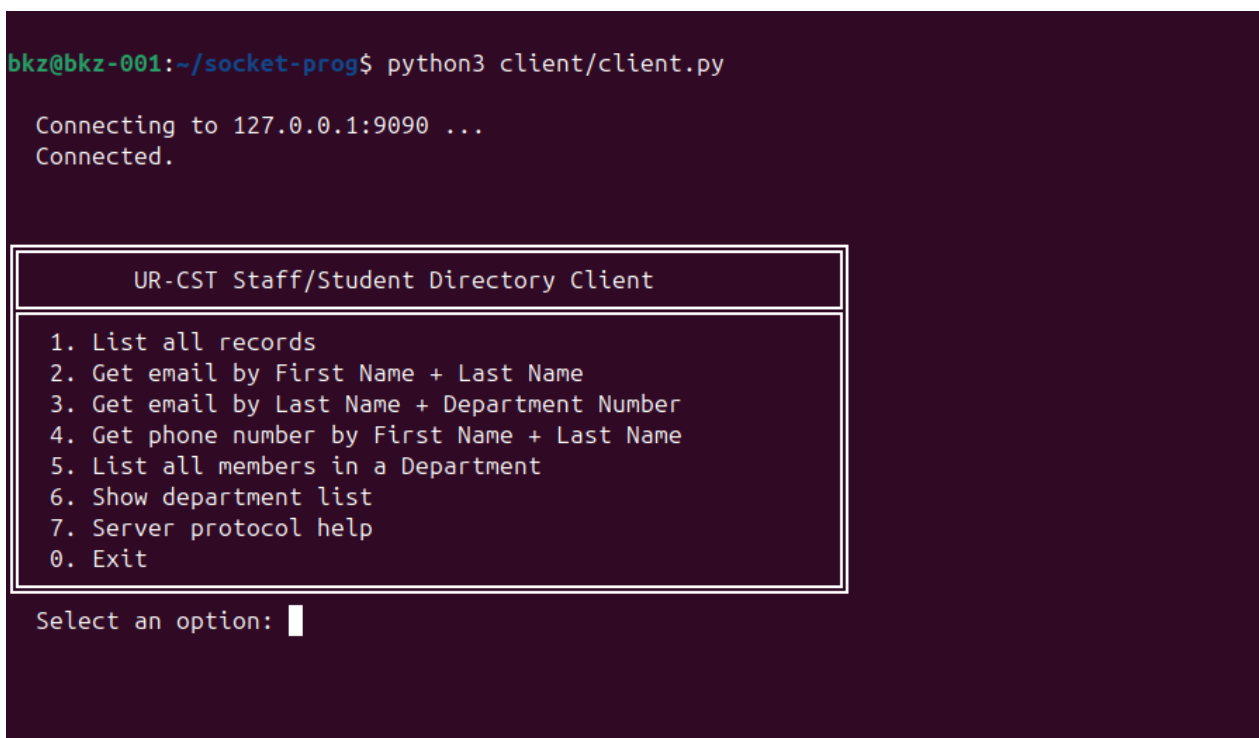
1. List all records
2. Get email by First Name + Last Name
3. Get email by Last Name + Department Number
4. Get phone number by First Name + Last Name
5. List all members in a Department
6. Show department list
7. Server protocol help
0. Exit

Select an option: 
```

SS2 — Left: server logs '[+] Connected'. Right: client prints 'Connected.' and shows the menu.

4.2 The Client Menu — Screenshot 3

Once connected, the client presents a menu of all supported query types. Each option maps to a different JSON action sent to the server.



```
bkz@bkz-001:~/socket-prog$ python3 client/client.py

Connecting to 127.0.0.1:9090 ...
Connected.

UR-CST Staff/Student Directory Client

1. List all records
2. Get email by First Name + Last Name
3. Get email by Last Name + Department Number
4. Get phone number by First Name + Last Name
5. List all members in a Department
6. Show department list
7. Server protocol help
0. Exit

Select an option: 
```

SS3 — The client menu listing all available query options (0–7).

5. The Application Protocol — JSON over TCP

Raw TCP is just a stream of bytes — it has no concept of messages or boundaries. Our application defines its own protocol on top of TCP:

- **Requests** are newline-terminated JSON objects sent by the client.

- **Responses** are plain-text strings sent by the server, terminated by a special sentinel token <<END>>.

The sentinel pattern is necessary because TCP is a stream protocol — the receiver does not know where one message ends and the next begins. By agreeing on a terminator, both sides know exactly when a full response has arrived.

5.1 Sending a Request

```
payload = {"action": "email_by_name", "first_name": "Jean", "last_name":
"Mugisha"} message = json.dumps(payload) + "\n"
sock.sendall(message.encode("utf-8"))
```

5.2 Receiving a Response

recv() may return less than the full response in a single call — TCP can split data across multiple packets. The client loops, accumulating chunks, until it sees the sentinel:

```
response = b"" while True: chunk = sock.recv(4096) if not chunk: raise
ConnectionResetError("Server closed the connection unexpectedly.") response +=
chunk if b"<<END>>" in response: break
```

6. Queries in Action

6.1 List All Records — Screenshot 4

Option 1 sends {"action": "list_all"}. The server iterates over every record in the database and returns a formatted table. This is the largest payload and best illustrates the recv-loop: the response spans multiple TCP segments before the sentinel arrives.

Select an option: 1

ALL RECORDS

Dept	Name	Phone	Email
Dept 1 (Computer Engineering)	Jean Mugisha	+250 781 234 567	jean.mugisha@ur.ac.rw
Dept 1 (Computer Engineering)	Alice Umutoni	+250 782 345 678	alice.umutoni@ur.ac.rw
Dept 1 (Computer Engineering)	Pierre Niyonsaba	+250 783 456 789	pierre.niyonsaba@ur.ac.rw
Dept 1 (Computer Engineering)	Marie Gakwaya	+250 784 567 890	marie.gakwaya@ur.ac.rw
Dept 1 (Computer Engineering)	Patrick Habimana	+250 785 678 901	patrick.habimana@ur.ac.rw
Dept 2 (Computer Science)	Diane Uwimana	+250 786 789 012	diane.uwimana@ur.ac.rw
Dept 2 (Computer Science)	Eric Nkurunziza	+250 787 890 123	eric.nkurunziza@ur.ac.rw
Dept 2 (Computer Science)	Grace Mukamana	+250 788 901 234	grace.mukamana@ur.ac.rw
Dept 2 (Computer Science)	Paul Bizimana	+250 789 012 345	paul.bizimana@ur.ac.rw
Dept 2 (Computer Science)	Solange Nyiramana	+250 780 123 456	solange.nyiramana@ur.ac.rw
Dept 3 (Information Systems)	Robert Nsanzimana	+250 781 234 901	robert.nsanzimana@ur.ac.rw
Dept 3 (Information Systems)	Yvonne Ingabire	+250 782 345 012	yvonne.ingabire@ur.ac.rw
Dept 3 (Information Systems)	Claude Hakizimana	+250 783 456 123	claudio.hakizimana@ur.ac.rw
Dept 3 (Information Systems)	Celine Uwitonze	+250 784 567 234	celine.uwitonze@ur.ac.rw
Dept 3 (Information Systems)	Joseph Ndayambaje	+250 785 678 345	joseph.ndayambaje@ur.ac.rw
Dept 4 (Information Technology)	Anita Mukashyaka	+250 786 789 456	anita.mukashyaka@ur.ac.rw
Dept 4 (Information Technology)	Denis Rutaganda	+250 787 890 567	denis.rutaganda@ur.ac.rw
Dept 4 (Information Technology)	Judith Nyirahabimana	+250 788 901 678	judith.nyirahabimana@ur.ac.rw
Dept 4 (Information Technology)	Samuel Gasana	+250 789 012 789	samuel.gasana@ur.ac.rw
Dept 4 (Information Technology)	Leonie Uwera	+250 780 123 890	leonie.uwera@ur.ac.rw

SS4 — Full directory table returned by the server for the 'list_all' action.

6.2 Email Lookup by First Name + Last Name — Screenshot 5a

Option 2 sends {"action": "email_by_name", "first_name": "...", "last_name": "..."}.

The server searches the database and returns a single email address — a minimal but

complete request-response cycle.

```
UR-CST Staff/Student Directory Client

1. List all records
2. Get email by First Name + Last Name
3. Get email by Last Name + Department Number
4. Get phone number by First Name + Last Name
5. List all members in a Department
6. Show department list
7. Server protocol help
0. Exit

Select an option: 2
First name: Jean
Last name: mugisha

Email for Jean Mugisha: jean.mugisha@ur.ac.rw
```

SS5a — Email retrieved by providing first and last name.

6.3 Email Lookup by Last Name + Department — Screenshot 5b

Option 3 sends `{"action": "email_by_lastname_dept", "last_name": "...", "dept_no": N}`. This is the second email-lookup method specified in the lab requirements — useful when only partial information is known. The `dept_no` field shows how integer parameters are carried inside the JSON.

```
0. Exit

Select an option: 3

Available Departments:
[1] Computer Engineering
[2] Computer Science
[3] Information Systems
[4] Information Technology
Last name: mugisha
Department number: 1

Email(s) for Mugisha in Computer Engineering:
Jean Mugisha: jean.mugisha@ur.ac.rw
```

SS5b — Email retrieved by last name and department number.

6.4 Phone Lookup by Name — Screenshot 6

Option 4 sends `{"action": "phone_by_name", ...}`. The same JSON structure as the email lookup but with a different action field, demonstrating how one protocol format can dispatch to multiple server behaviours.

```
0. Exit

Select an option: 4
First name: Jean
Last name: mugisha

Phone for Jean Mugisha: +250 781 234 567
```

6.5 List All Members in a Department — Screenshot 7

Option 5 sends `{"action": "list_by_dept", "dept_no": N}`. The client only needs to supply a department number; the server returns a formatted table of every person in that department.

```

Select an option: 5

Available Departments:
  [1] Computer Engineering
  [2] Computer Science
  [3] Information Systems
  [4] Information Technology
Department number: 2

Members of Computer Science (Dept 2):
  Name | Phone | Email
  -----+-----+-----
  Diane Uwimana | +250 786 789 012 | diane.uwimana@ur.ac.rw
  Eric Nkurunziza | +250 787 890 123 | eric.nkurunziza@ur.ac.rw
  Grace Mukamana | +250 788 901 234 | grace.mukamana@ur.ac.rw
  Paul Bizimana | +250 789 012 345 | paul.bizimana@ur.ac.rw
  Solange Nyiramana | +250 780 123 456 | solange.nyiramana@ur.ac.rw

```

SS7 — All members of a selected department returned by the server.

6.6 Server Protocol Help — Screenshot 8

Option 7 sends `{"action": "help"}` and receives the full protocol reference document. This is a real-world pattern: a self-describing protocol that documents itself at runtime so any client can discover what actions are available without reading source code.

```
Select an option: 7

UR-CST Directory Server – Protocol Reference
=====

All requests are newline-terminated JSON objects sent over TCP.
All responses are plain text terminated by a "< <END> >" sentinel line (no spaces).

ACTION: help

Request : {"action": "help"}
Response: This help text.

ACTION: list_all

Request : {"action": "list_all"}
Response: Formatted table of every record in the database.
Example response (truncated):
  ALL RECORDS
  Dept 1 (Computer Science ...) | Jean      Mugisha          | +250 781 234 567   | jean.mugisha@ur.ac.rw
  ...

ACTION: email_by_name

Request : {"action": "email_by_name", "first_name": "<str>", "last_name": "<str>"}
Required: first_name, last_name
Response (success): "Email for <First> <Last>: <email>"
Response (not found): "ERROR: No record found for '<First> <Last>'."
Response (missing field): "ERROR: 'first_name' and 'last_name' are required."

ACTION: email_by_lastname_dept

Request : {"action": "email_by_lastname_dept", "last_name": "<str>", "dept_no": <int>}
Required: last_name, dept_no (integer 1-7)
Response (success): "Email(s) for <Last> in <Dept Name>:\n <First> <Last>: <email>\n ..."
Response (not found): "ERROR: No record found for last name '<Last>' in dept <N>."
Response (bad dept): "ERROR: Department number <N> does not exist."

ACTION: phone_by_name

Request : {"action": "phone_by_name", "first_name": "<str>", "last_name": "<str>"}
Required: first_name, last_name
Response (success): "Phone for <First> <Last>: <phone>"
Response (not found): "ERROR: No record found for '<First> <Last>'."
```

SS8 — The server's built-in protocol reference returned over the socket.

7. Error Handling

Robust socket applications must handle errors at two levels: **application-level errors** (e.g., a lookup that finds no match) and **network-level errors** (e.g., the server disappears while the client is running).

7.1 Record Not Found — Screenshot 9

When a query finds no matching record, the server sends back a plain-text error string beginning with `ERROR:`. The response travels over the same socket connection using the same sentinel-terminated protocol — errors are just another type of response.

```
0. Exit
Select an option: 2
First name: John
Last name: Doe

ERROR: No record found for 'John Doe'.
```

SS9 — The server returns an `ERROR` message when no matching record exists.

7.2 Connection Refused — Screenshot 10

If the client tries to connect while the server is not running, `sock.connect()` raises `ConnectionRefusedError` (or `OSError` on timeout). The client catches this and prints a clear message rather than crashing with a raw traceback.

```
Select an option: 2
First name: Mikel
Last name: Arteta

[!] Connection error: Server closed the connection unexpectedly.
bkz@bkz-001:~/socket-prog$
```

SS10 — Client reports connection refused when the server is not running.

8. Handling Multiple Clients — Threading

One of the lab requirements is that the server must accept several client connections at a time. A naive server that handles one client at a time inside the accept loop would block all other clients until the first one disconnects.

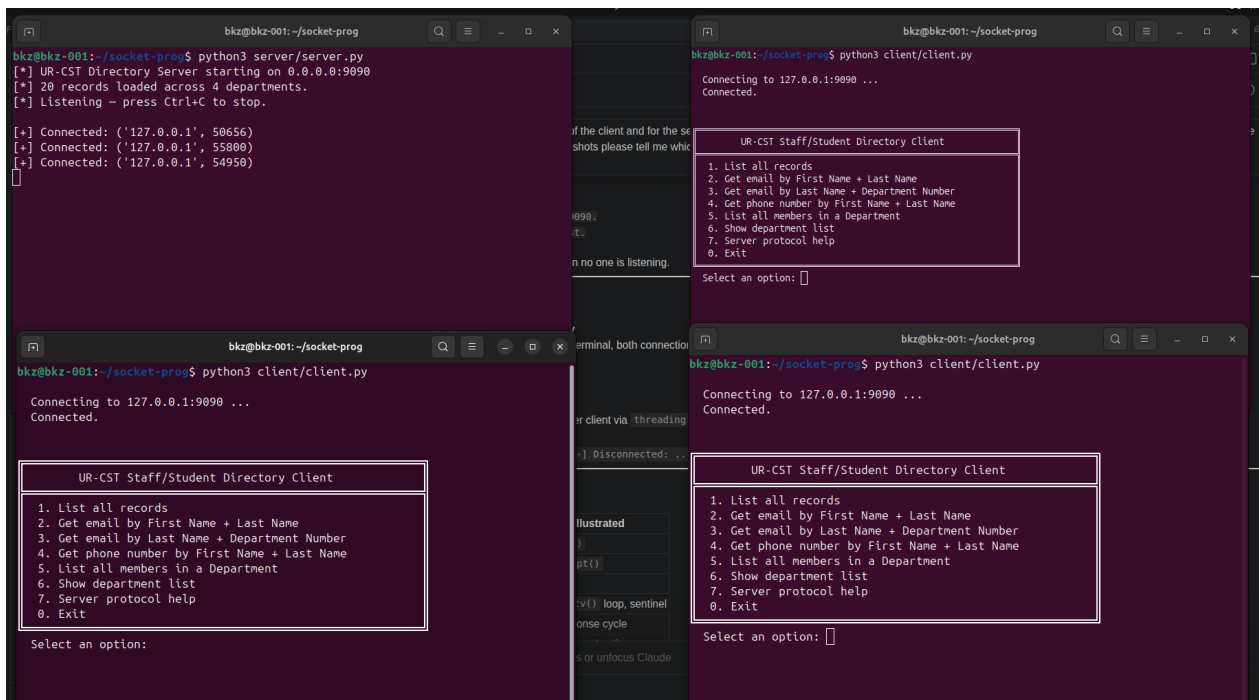
Our server solves this with **threading**: every time `accept()` returns a new connection, a dedicated thread is spawned to handle that client independently. The main thread immediately returns to waiting for the next connection.

```
while True: conn, addr = srv.accept() t = threading.Thread(target=client_handler,
args=(conn, addr), daemon=True) t.start()
```

The `daemon=True` flag means the threads are automatically killed when the main program exits, so the server shuts down cleanly on `Ctrl+C`.

8.1 Two Clients Connected Simultaneously — Screenshot 11

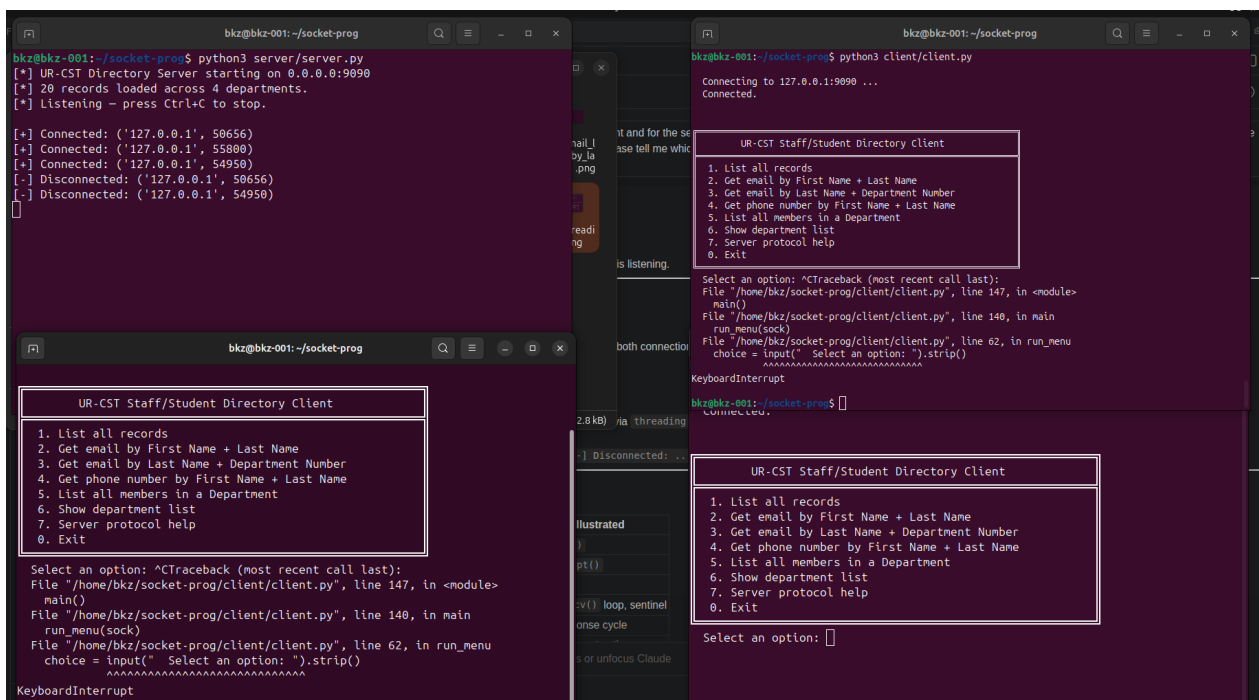
The screenshot below shows the server terminal with two separate clients connected at the same time. Each client has a different ephemeral port number, confirming they are distinct TCP connections being served by separate threads.



SS11 — Server logs two simultaneous connections, each handled by its own thread.

8.2 Client Disconnect Logged on the Server — Screenshot 12

When a client exits (option 0), the TCP connection is closed. The server's `client_handler` detects this because `recv()` returns empty bytes, the thread ends, and the disconnect is logged. This demonstrates the full lifecycle of a socket: *connect* → *exchange data* → *close*.



SS12 — Server logs '[-] Disconnected' when a client closes the connection.

9. Summary

This lab demonstrated the complete lifecycle of a TCP socket-based client-server application using Python. The key concepts covered are:

Concept	Where demonstrated
socket() / SOCK_STREAM	server.py:264, client.py:138
bind() + listen()	server.py:265-267 — server binds to 0.0.0.0:9090
connect()	client.py:136 — client dials the server
accept()	server.py:269 — server blocks until a client arrives
sendall() / recv()	client.py:34,38 — full send + chunked receive loop
Message framing (sentinel)	<<END>> token marks end of every server response
JSON application protocol	All requests are JSON; responses are plain text
Threading	server.py:271 — one thread per connected client
Error handling	SS9 (not found), SS10 (refused), SS12 (disconnect)

Through building and running this application, we saw that socket programming requires careful attention to **framing** (knowing where messages begin and end), **concurrency** (serving multiple clients without blocking), and **error handling** at both the network and application layer. These are the same principles that underpin every distributed system, from web servers to database clusters.